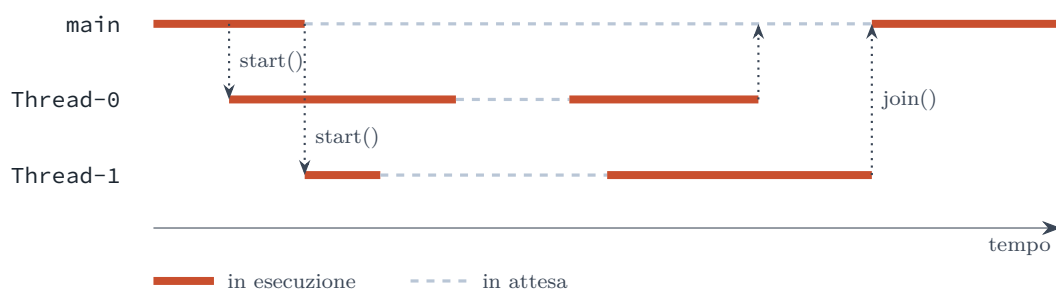


Thread e concorrenza

Dal processo alla JVM, executor, sincronizzazione
e la coda degli eventi di Swing



prof. Massimiliano Redolfi

Università degli Studi di Brescia · Ingegneria Informatica

massimiliano.redolfi@unibs.it

Versione 0.1.1

Quest'opera è distribuita con licenza [Creative Commons](https://creativecommons.org/licenses/by-nc-sa/4.0/)
"Attribuzione – Non commerciale – Condividi allo stesso
modo 4.0 Internazionale".



Indice

4 Thread e concorrenza	1
4.1 Processi e thread nel sistema operativo	1
4.2 Il primo thread	4
4.3 Executor: la via semplice	8
4.4 Valori di ritorno: <code>Callable</code> e <code>Future</code>	10
4.5 Concorrenza: quando i thread condividono	14
4.6 Gli strumenti di <code>java.util.concurrent</code>	19
4.7 Esecuzione differita e periodica	23
4.8 Thread e stream paralleli	24
4.9 Thread e Swing: la coda degli eventi	25
4.10 Thread virtuali	31
4.11 Riepilogo	32
4.12 Esercizi	32
Soluzioni	34

Thread e concorrenza

Dal processo alla JVM, executor, sincronizzazione e la coda degli eventi di Swing.

Il punto di partenza è il programma come lo abbiamo sempre scritto: una sequenza di istruzioni eseguita da un unico flusso, che fa una cosa alla volta e, quando aspetta (un calcolo lungo, un file, la rete), aspetta e basta. Il punto di arrivo è un programma in cui più flussi lavorano insieme: uno conta, uno legge, uno risponde all'utente, e nessuno dei tre tiene fermi gli altri. In mezzo c'è tutto ciò che rende questa idea semplice da enunciare e delicata da realizzare: che cos'è un thread per il sistema operativo e per la JVM, come lo si avvia e come lo si ferma, come si distribuisce il lavoro senza creare un thread per ogni compito, che cosa succede quando due thread toccano lo stesso dato e con quali strumenti si evita. L'ultima parte applica il tutto al caso che ogni programma con un'interfaccia grafica incontra: Swing lavora su un solo thread, e un calcolo lungo al posto sbagliato congela la finestra.

Lambda expression e stream sono dati per noti: sono l'argomento di dispense a parte, e qui li usiamo come linguaggio corrente.

4.1 Processi e thread nel sistema operativo

Prima di scrivere `new Thread(...)` conviene sapere che cosa si sta chiedendo al sistema operativo, perché il thread di Java è un'astrazione sottile sopra un oggetto che esiste già nel kernel, con i suoi costi e le sue regole.

4.1.1 Il processo

Un *processo* è un programma in esecuzione: il sistema operativo gli assegna uno spazio di indirizzamento proprio (codice, dati, heap, stack), una tabella dei file aperti, un identificatore e dei diritti. L'isolamento è la sua proprietà fondamentale: un processo non può leggere la memoria di un altro, e se va in errore il sistema lo termina senza che gli altri se ne accorgano. Il prezzo di questa protezione è che crearne uno costa (si copia o si prepara uno spazio di indirizzamento intero) e che due processi, per scambiarsi un dato, devono passare da un canale esterno: un file, una pipe, un socket.

4.1.2 Il thread

Un *thread* (filo) è un flusso di esecuzione *dentro* un processo. Un processo ne ha almeno uno, quello che esegue `main`; può crearne altri, e tutti condividono lo stesso spazio di indirizzamento. Ciò che ogni thread ha di suo è il minimo indispensabile per avanzare in modo indipendente:

Privato del thread	Condiviso dal processo
stack (variabili locali, chiamate in corso)	heap (tutti gli oggetti)
program counter (l'istruzione a cui è arrivato)	codice del programma
registri del processore	variabili globali e statiche
stato (pronto, in esecuzione, bloccato)	file aperti, socket, finestre

La colonna di destra è la ragione per cui i thread esistono e, insieme, la fonte di ogni loro problema. Esistono perché creare un thread costa molto meno di un processo (uno stack e qualche registro, non uno spazio di indirizzamento) e perché due thread si scambiano dati semplicemente leggendo e scrivendo gli stessi oggetti. Sono un problema per lo stesso motivo: se due thread scrivono lo stesso oggetto nello stesso momento, nessuno li protegge. La sezione 4.5 è dedicata a questo.

4.1.3 Scheduler e context switch

I thread pronti a eseguire sono quasi sempre più dei core disponibili. Lo *scheduler* del sistema operativo decide quale thread gira su quale core e per quanto: a ogni thread tocca una *fetta di tempo* (qualche millisecondo), scaduta la quale il sistema lo sospende, ne salva i registri e carica quelli di un altro. Questo scambio è il *context switch*: costa poco, ma non niente, e con migliaia di thread diventa una parte visibile del lavoro. La sospensione è *preemptive*: il thread non deve collaborare, e non può impedirla. Da qui la prima regola mentale della programmazione con i thread: fra due istruzioni qualsiasi può passare un tempo arbitrario, durante il quale altri thread fanno quello che vogliono.

Vale la pena distinguere due parole che il linguaggio comune confonde. *Concorrenza* è avere più flussi in corso nello stesso intervallo di tempo, anche su un solo core che li alterna: il programma è corretto o sbagliato indipendentemente dal numero di core. *Parallelismo* è eseguirli davvero nello stesso istante su core diversi: è una questione di velocità, e la ottiene solo chi ha già risolto la concorrenza.

Dal punto di vista dello scheduler un thread è in uno di tre stati: *in esecuzione* su un core; *pronto*, cioè vorrebbe eseguire e aspetta un core; *bloccato*, cioè aspetta qualcosa che non è un core (un dato dal disco, un timer, un lock tenuto da altri) e fino ad allora non viene nemmeno considerato. Un thread bloccato non consuma processore: è per questo che un programma con cento thread in attesa di rete costa quasi nulla, mentre dieci thread che calcolano saturano dieci core.

4.1.4 Come fanno i sistemi moderni

I tre sistemi su cui gira il materiale del corso realizzano i thread in modi diversi nei dettagli e uguali nella sostanza: il kernel conosce i thread e li schedula direttamente, un thread del linguaggio corrisponde a un thread del kernel (modello *uno a uno*), e ogni thread ha uno stack riservato in anticipo.

	Linux	macOS	Windows
che cos'è un thread	un <i>task</i> come il processo, creato con <code>clone</code> condividendo memoria e file; il processo è il gruppo dei suoi task	un thread Mach del kernel XNU; sopra, la libreria <code>pthread</code>	l'unità di esecuzione del kernel; il processo è il contenitore di risorse
chiamata di sistema	<code>clone</code> , tramite <code>pthread_create</code> (NPPL)	<code>pthread_create</code> , che chiama il kernel Mach	<code>CreateThread</code>
scheduler	CFS fino alla 6.5, EEVDF dalla 6.6: ogni thread riceve una quota equa di tempo	a priorità con <i>classi di qualità del servizio</i> (interattivo, utente, utility, sfondo) che pesano anche sulla scelta fra core veloci ed efficienti	a priorità (32 livelli) con innalzamento temporaneo per i thread interattivi e quelli appena sbloccati
pool di sistema	nessuno: le librerie se lo fanno	Grand Central Dispatch: code di compiti servite da un pool del sistema	thread pool nativo e <i>fiber</i> cooperativi
stack per thread	8 MB virtuali per default (<code>ulimit -s</code>)	512 KB (8 MB il principale)	1 MB

Il punto comune è che oggi tutti schedulano thread, non processi: il processo è un contenitore di risorse, il thread è ciò che gira. Un'altra cosa che la tabella mostra è che uno stack si riserva per ogni thread; è memoria virtuale, consumata davvero solo quando il thread la usa, ma è il motivo per cui “un thread per ogni cosa” smette di funzionare attorno a qualche migliaio di thread.

Per vedere il livello sotto Java basta un programma C, visto che anche quello fa parte del corso. La libreria `pthread` è la stessa su Linux e macOS:

```
/* thread.c: compilare con gcc thread.c -o thread -lpthread */
#include <pthread.h>
#include <stdio.h>

void *lavoro(void *arg) {
    printf("ciao da un altro thread (%s)\n", (char *) arg);
    return NULL;
}

int main(void) {
    pthread_t t;
    pthread_create(&t, NULL, lavoro, "T1"); /* avvia: clone() sotto */
    printf("ciao dal thread principale\n");
    pthread_join(t, NULL); /* aspetta che finisca */
    return 0;
}
```

Le due stampe possono uscire in un ordine o nell'altro: dopo `pthread_create` ci sono due thread pronti, e chi gira per primo lo decide lo scheduler. Lo stesso programma in Java, nella sezione 4.2, farà esattamente le stesse chiamate, nascoste dietro due metodi.

4.1.5 L'astrazione di Java

In Java un thread è un oggetto della classe `java.lang.Thread`. Quando si chiama `start()` la JVM chiede al sistema operativo un thread nativo (`pthread_create` o `CreateThread`) e vi lega l'oggetto: da quel momento il thread Java e il thread del kernel sono la stessa cosa, con lo stesso ciclo di vita, lo stesso stack (un megabyte per default, regolabile con `-Xss`) e lo stesso scheduler. È il *thread di piattaforma*; l'uno-a-uno vale per tutti i thread che incontreremo fino alla sezione 4.10, dove Java 21 introduce un secondo tipo.

Il programma non è mai solo. Già un `main` che stampa una riga gira in una JVM con una ventina di thread: `main`, i thread del garbage collector, quelli del compilatore JIT, `Reference Handler`, `Finalizer`, `Signal Dispatcher`. Si vedono con il debugger di Eclipse (vista *Debug*, sospendendo il programma) o con lo strumento `jstack` del JDK, che stampa lo stack di ogni thread di una JVM in esecuzione: è anche il modo più diretto per capire che cosa sta facendo un programma bloccato.

La classe `Thread` descrive lo stato con l'enumerazione `Thread.State`, che raffina i tre stati del sistema operativo:

Stato Java	Che cosa significa	Per il SO
NEW	creato, <code>start()</code> non ancora chiamato	non esiste
RUNNABLE	in esecuzione, pronto, o in attesa di I/O	in esecuzione / pronto / bloccato
BLOCKED	aspetta di entrare in un blocco <code>synchronized</code>	bloccato
WAITING	<code>wait()</code> , <code>join()</code> , <code>park()</code> senza scadenza	bloccato
TIMED_WAITING	<code>sleep(ms)</code> , <code>wait(ms)</code> , <code>join(ms)</code>	bloccato
TERMINATED	<code>run()</code> è finito, bene o male	non esiste più

Due dettagli che tornano utili. La JVM termina quando finisce l'ultimo thread *non daemon*: un thread marcato `setDaemon(true)` prima di `start()` è di servizio e non tiene vivo il programma (così sono i thread del garbage collector). Le priorità (`setPriority`, da 1 a 10) sono un suggerimento allo scheduler che su Linux per default viene ignorato e altrove conta poco: un programma corretto non dipende mai dalle priorità.

4.2 Il primo thread

4.2.1 Compito e lavoratore

Java separa due cose: *che cosa* va eseguito e *chi* lo esegue. Il compito è un `Runnable`, l'interfaccia funzionale con il solo metodo `void run()`; il lavoratore è un `Thread`, che riceve il compito nel costruttore.

```
Runnable task = () -> {  
    String nome = Thread.currentThread().getName();  
    System.out.println("Ciao, sono " + nome);  
};  
  
task.run(); // 1: lo eseguo io  
  
Thread thread = new Thread(task);  
thread.start(); // 2: lo esegue un altro thread  
  
System.out.println("Fatto!"); // 3
```

```
Ciao, sono main  
Fatto!  
Ciao, sono Thread-0
```

La prima riga non sorprende: `task.run()` è una chiamata di metodo come le altre, eseguita dal thread `main`. Le due successive sì: “Fatto!” esce *prima* del saluto del secondo thread. `start()` chiede al sistema un nuovo thread e ritorna subito, senza aspettare che quello parta; `main` prosegue e stampa; nel frattempo `Thread-0` viene schedato, esegue `run()` e stampa a sua volta. L’ordine delle ultime due righe non è garantito: su una macchina carica, o con un’altra JVM, potrebbero invertirsi. È il primo insegnamento pratico: due thread non hanno un ordine reciproco, a meno che il programma non lo imponga.

Attenzione 4.1

Chiamare `thread.run()` invece di `thread.start()` compila, gira, e non crea nessun thread: `run` è un metodo normale e viene eseguito da chi lo chiama. È l’errore più frequente del principiante e non dà nessun segnale, se non che il programma resta sequenziale.

Un compito con dei dati ha bisogno di un posto dove tenerli: una classe che implementa `Runnable`. Questa, che stampa `nStep` righe a intervalli di `delay` millisecondi, servirà da esempio per tutta la prima parte:

```

public class MyTask implements Runnable {
    private final String nome;
    private final int nStep, delay;

    public MyTask(String nome, int nStep, int delay) {
        this.nome = nome; this.nStep = nStep; this.delay = delay;
    }

    public void run() {
        for (int i = 0; i < nStep; i++) {
            System.out.printf("%s, passo %2d [%s]%n",
                              nome, i, Thread.currentThread().getName());
            try {
                Thread.sleep(delay);
            } catch (InterruptedException e) {
                return; // ci hanno chiesto di smettere
            }
        }
    }
}

```

```

new Thread(new MyTask("A", 5, 30)).start();
new Thread(new MyTask("B", 5, 20)).start();
System.out.println("main ha finito");

```

```

main ha finito
A, passo 0 [Thread-0]
B, passo 0 [Thread-1]
B, passo 1 [Thread-1]
A, passo 1 [Thread-0]
B, passo 2 [Thread-1]
...

```

main finisce subito, ma il programma no: la JVM resta viva finché A e B, che non sono daemon, non hanno completato il loro run. Le righe si intrecciano secondo i tempi dei due sleep e le decisioni dello scheduler; eseguendo due volte si ottengono due intrecci diversi.

4.2.2 Aspettare: sleep e join

Thread.sleep(ms) mette il thread corrente in TIMED_WAITING per almeno ms millisecondi: non consuma processore, e *almeno* vuol dire che il risveglio dipende dallo scheduler. t.join() mette il thread corrente in attesa che t termini: è il modo per imporre un ordine, quando serve.

```

Thread a = new Thread(new MyTask("A", 5, 30));
a.start();
a.join(); // main aspetta A
System.out.println("A ha finito, ora tocca a main");

```

Entrambi dichiarano InterruptedException, un'eccezione *checked* che il compilatore obbliga a gestire. Non è pedanteria: è l'intero meccanismo con cui in Java si chiede a un

thread di smettere.

4.2.3 Fermare un thread: l'interruzione

Un thread non si può uccidere dall'esterno (c'era un `stop()`, deprecato da decenni e oggi reso inoperante, perché lasciava gli oggetti a metà). Si può solo *chiederli* di fermarsi, e il thread deve collaborare. La richiesta è `t.interrupt()`, che alza una bandiera nel thread `t`; il thread la vede in due modi:

- se sta calcolando, controllando `Thread.currentThread().isInterrupted()` a intervalli ragionevoli, per esempio a ogni giro di ciclo;
- se è in attesa (`sleep`, `join`, `wait`, una coda bloccante), l'attesa viene interrotta e lancia `InterruptedException`, abbassando la bandiera.

Ecco perché `MyTask` esce dal ciclo con `return` nel `catch`: un `interrupt()` ricevuto durante lo `sleep` è la richiesta di smettere, e il compito la rispetta. Un ciclo di solo calcolo, senza attese, deve invece controllare la bandiera da sé:

```
public void run() {  
    while (!Thread.currentThread().isInterrupted()) {  
        faiUnPassoDiCalcolo();  
    }  
}
```

Attenzione 4.2

Il `catch` (`InterruptedException e`) { } vuoto, quello che Eclipse genera e che si vede in troppi programmi, ingoia la richiesta: la bandiera è stata abbassata dall'eccezione, il ciclo riprende e il thread non si fermerà mai. Nel `catch` si fa una di due cose: si esce (`return`), oppure, se non si è il proprietario del ciclo, si rialza la bandiera con `Thread.currentThread().interrupt()` e si lascia che sia chi sta sopra a decidere.

Negli esempi che seguono una pausa serve spesso solo a rendere visibile l'intreccio dei thread; per non ripetere il `try` ogni volta useremo un metodo di comodo che applica la seconda regola:

```
static void pausa(long ms) {  
    try {  
        Thread.sleep(ms);  
    } catch (InterruptedException e) {  
        Thread.currentThread().interrupt(); // rialza la bandiera e ritorna  
    }  
}
```

4.2.4 Lambda e stato

Un `Runnable` è un'interfaccia funzionale, quindi un compito può essere una lambda: `new Thread(() -> ...)`. Vale la regola delle variabili catturate: la lambda vede le variabili locali del contesto solo se sono *effectively final*. Un compito che deve *modificare* uno stato (un contatore, un risultato) non può quindi tenerlo in una variabile locale del metodo che lo crea: lo stato va in un oggetto, come i campi di `MyTask`, oppure in un contenitore pensato per essere condiviso, come quelli della sezione 4.6. Ed è qui che si vede la differenza fra estendere `Thread` e implementare `Runnable`: la seconda tiene insieme il compito e il suo stato in un oggetto che non sa, né deve sapere, da chi sarà eseguito. Nella sezione seguente il lavoratore smette di essere un `Thread` creato a mano, e i compiti scritti come `Runnable` restano identici.

4.3 Executor: la via semplice

4.3.1 Perché non un thread per ogni compito

Con `new Thread(task).start()` ogni compito costa un thread del sistema operativo: uno stack riservato, una chiamata al kernel, un context switch in più da gestire. Per dieci compiti è irrilevante; per diecimila è un problema di memoria e di tempo, e per un flusso continuo di piccoli compiti (le richieste di un server, gli eventi di un'interfaccia) è il modo sbagliato di pensare. Il modello giusto è quello della tabella in sezione 4.1: pochi thread che vivono a lungo e una coda di compiti da cui pescano. In Java si chiama *executor*, sta nel package `java.util.concurrent`, e separa definitivamente il *che cosa* dal *chi*.

4.3.2 ExecutorService e le factory di Executors

```
ExecutorService executor = Executors.newSingleThreadExecutor();
executor.submit(new MyTask("A", 3, 30));
executor.submit(new MyTask("B", 3, 20));
executor.submit(new MyTask("C", 3, 10));
System.out.println("Fatto!");
executor.shutdown();
```

```
Fatto!
A, passo 0 [pool-1-thread-1]
A, passo 1 [pool-1-thread-1]
A, passo 2 [pool-1-thread-1]
B, passo 0 [pool-1-thread-1]
B, passo 1 [pool-1-thread-1]
...
```

`submit` accoda il compito e ritorna subito; un unico thread, `pool-1-thread-1`, esegue i tre compiti *in sequenza*, nell'ordine di arrivo. Cambiando una riga cambia il lavoratore ma non i compiti:

```
ExecutorService executor = Executors.newCachedThreadPool();
```

```
Fatto!
A, passo 0 [pool-1-thread-1]
B, passo 0 [pool-1-thread-2]
C, passo 0 [pool-1-thread-3]
C, passo 1 [pool-1-thread-3]
B, passo 1 [pool-1-thread-2]
...
```

Le factory della classe `Executors` coprono i casi ricorrenti:

Factory	Che cosa fa
<code>newSingleThreadExecutor()</code>	un thread, compiti in sequenza: serializza senza bloccare chi li manda
<code>newFixedThreadPool(n)</code>	n thread fissi e una coda illimitata: il caso tipico per il calcolo
<code>newCachedThreadPool()</code>	crea thread quando servono, li riusa, li elimina dopo un minuto di inattività: per molti compiti brevi che aspettano (I/O)
<code>newScheduledThreadPool(n)</code>	come il pool fisso, ma i compiti possono essere ritardati o periodici (sezione 4.7)
<code>newWorkStealingPool()</code>	un <code>ForkJoinPool</code> con tanti thread quanti i core, in cui ogni thread ha la sua coda e ruba agli altri quando ha finito: lo stesso usato dagli stream paralleli

`ExecutorService` ha due metodi per consegnare un compito. `execute(Runnable)` lo esegue e basta: un'eccezione non catturata nel compito termina il thread del pool (che viene rimpiazzato) e viene stampata dal gestore di default. `submit(Runnable)` e `submit(Callable)` restituiscono un `Future`, di cui parla la sezione 4.4: con `submit` l'eccezione *non* viene stampata ma conservata nel `Future`, e chi non lo interroga non la vedrà mai. È una trappola classica: un compito che fallisce in silenzio.

4.3.3 Spegnerne un executor

Il primo programma con un executor finisce così:

```
ExecutorService executor = Executors.newSingleThreadExecutor();
executor.submit(() -> System.out.println("Ciao da "
    + Thread.currentThread().getName()));
```

```
Ciao da pool-1-thread-1
-                               <- e il programma non termina
```

I thread di un pool non sono daemon e restano in attesa di altri compiti, per sempre: la JVM, di conseguenza, non esce. Un executor va spento. `shutdown()` chiude l'ingresso: i compiti già in coda vengono completati, quelli nuovi rifiutati, e i thread terminano quando la coda si svuota. `shutdownNow()` è la versione brusca: svuota la coda, interrompe (con `interrupt()`, sezione 4.2) i compiti in corso e restituisce la lista di quelli mai partiti. `awaitTermination(t, unit)` blocca il chiamante finché tutto non è finito, o fino allo scadere del tempo.

Ricetta: spegnere un executor

```

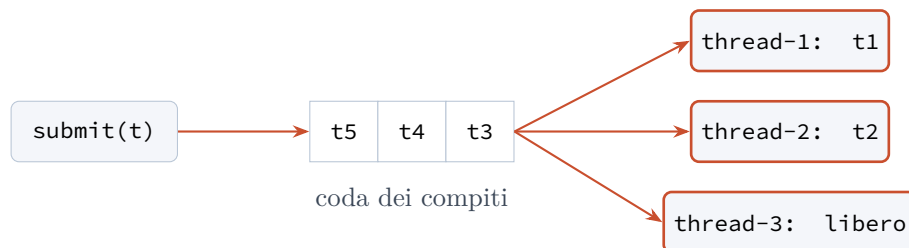
executor.shutdown(); // 1. niente nuovi compiti
try {
    if (!executor.awaitTermination(5, TimeUnit.SECONDS)) {
        executor.shutdownNow(); // 2. troppo lenti: interrompi
    }
} catch (InterruptedException e) {
    executor.shutdownNow(); // 3. interrotti: interrompi tutti
    Thread.currentThread().interrupt();
}

```

Da Java 19 `ExecutorService` è `AutoCloseable` e `close()` fa `shutdown` seguito da un'attesa illimitata: in un metodo che crea l'executor, lo usa e lo chiude, `try (ExecutorService ex = ...) { ... }` **JAVA 19** è la forma più breve e corretta.

4.3.4 Dentro un pool

Un pool è una coda più un gruppo di thread che eseguono tutti lo stesso ciclo: estrai un compito, esegilo, ricomincia. Chi chiama `submit` non aspetta nessuno; chi esegue non sa chi ha mandato il compito.



Quanti thread? Dipende da che cosa fanno i compiti. Se *calcolano* (CPU-bound), più thread dei core non servono: si alternano sugli stessi core pagando i context switch; `Runtime.getRuntime().availableProcessors()` dà il numero giusto. Se *aspettano* (I/O-bound: rete, disco, un altro servizio) la maggior parte del tempo sono bloccati e non occupano un core: possono essere molti di più, e il limite è la memoria degli stack. Il pool con *n* thread fissi e la coda illimitata è la scelta di default per il calcolo; il pool *cached* per i compiti brevi che aspettano; e per l'I/O massiccio Java 21 offre un'alternativa migliore di entrambi (sezione 4.10).

4.4 Valori di ritorno: **Callable** e **Future**

4.4.1 Da **Runnable** a **Callable**

`Runnable.run()` non restituisce niente e non può lanciare eccezioni checked: va bene per un compito che agisce (stampa, aggiorna, scrive), non per uno che *calcola* un risultato. Per questo esiste `Callable<V>`, l'altra interfaccia funzionale dei compiti:

```
@FunctionalInterface
public interface Callable<V> {
    V call() throws Exception;
}
```

Il compito guida di questa parte conta i numeri primi in un intervallo. Non è un problema utile in sé, ma ha tre proprietà che lo rendono perfetto per i thread: è puro calcolo, si divide in pezzi indipendenti e il lavoro non è uniforme (i numeri grandi costano più dei piccoli).

```
public class PrimeTester implements Callable<Long> {
    private final long start, end;

    public PrimeTester(long start, long end) {
        this.start = start; this.end = end;
    }

    public Long call() {
        long n = 0;
        for (long i = start; i <= end; i++)
            if (isPrime(i)) n++;
        return n;
    }

    public static boolean isPrime(long n) {
        if (n < 2) return false;
        if (n % 2 == 0) return n == 2;
        for (long d = 3; d * d <= n; d += 2)
            if (n % d == 0) return false;
        return true;
    }
}
```

4.4.2 Future: un risultato che arriverà

`submit(Callable<V>)` restituisce subito un `Future<V>`: una promessa del risultato, che il compito riempirà quando avrà finito.

```
ExecutorService executor = Executors.newFixedThreadPool(2);
Future<Long> fa = executor.submit(new PrimeTester(1, 50_000_000));
Future<Long> fb = executor.submit(new PrimeTester(50_000_001, 100_000_000));

System.out.println("compiti avviati, main e' libero");
long totale = fa.get() + fb.get();           // qui main aspetta
System.out.println("primi fino a 10^8: " + totale);
executor.shutdown();
```

```
compiti avviati, main e' libero
primi fino a 10^8: 5761455
```

`get()` è il punto di sincronizzazione: blocca il chiamante finché il risultato non c'è, poi lo restituisce. Il resto dell'interfaccia serve a non bloccarsi alla cieca: `isDone()` risponde subito;

`get(t, unit)` aspetta al massimo `t` e poi lancia `TimeoutException`; `cancel(true)` interrompe il compito se è in corso, e lo toglie dalla coda se non è ancora partito. Le eccezioni del compito seguono il risultato: se `call()` lancia, `get()` rilancia un'`ExecutionException` che *incarta* l'eccezione originale, recuperabile con `getCause()`. `get()` dichiara anche `InterruptedException`, perché è un'attesa come `sleep` e `join`, e vale la stessa regola.

4.4.3 `invokeAll` e `invokeAny`

Per una collezione di compiti ci sono due scorciatoie. `invokeAll` li esegue tutti e restituisce la lista dei `Future`, già completati:

```
List<Callable<String>> compiti = List.of(
    () -> "alfa", () -> "beta", () -> "gamma");

List<String> risultati = new ArrayList<>();
for (Future<String> f : executor.invokeAll(compiti))
    risultati.add(f.get());           // non blocca: sono già finiti
```

`invokeAny` esegue i compiti e restituisce il risultato del *primo* che termina, annullando gli altri: utile quando più strade portano allo stesso risultato (tre server che rispondono alla stessa domanda) e si vuole la più veloce.

4.4.4 Esempio guida: contare i primi fino a 10^8

Dividiamo l'intervallo in `nTask` pezzi, li consegniamo a un pool di `nThread` thread e sommiamo i risultati. Il tempo lo misura un cronometro di poche righe:

```
public class Chrono {
    private final long t0 = System.nanoTime();
    public long millis() { return (System.nanoTime() - t0) / 1_000_000; }
}
```

```

long max = 100_000_000L;
int nTask = 64;
int nThread = Runtime.getRuntime().availableProcessors();
ExecutorService executor = Executors.newFixedThreadPool(nThread);
List<Future<Long>> risultati = new ArrayList<>();
Chrono chrono = new Chrono();

long perTask = max / nTask;
for (int i = 0; i < nTask; i++) {
    long da = i * perTask + 1;
    long a = (i == nTask - 1) ? max : (i + 1) * perTask;
    risultati.add(executor.submit(new PrimeTester(da, a)));
}

long nPrimi = 0;
for (Future<Long> f : risultati)
    nPrimi += f.get();

System.out.printf("%d primi in %d ms con %d thread%n",
    nPrimi, chrono.millis(), nThread);
executor.shutdown();

```

Su un portatile con 16 core (12 veloci e 4 efficienti), facendo variare nThread:

thread	tempo	accelerazione
1	14,4 s	1,0
2	7,1 s	2,0
4	3,9 s	3,7
8	2,0 s	7,1
12	1,5 s	9,9
16	1,4 s	10,6
32	1,3 s	11,3

Fino a 8 thread l'accelerazione è quasi lineare: ogni thread ha il suo core veloce. Oltre, i core efficienti rendono meno e i thread in più dei core non aggiungono niente, perché si alternano sugli stessi core. Il limite non è Java: è la *legge di Amdahl*. Se una frazione p del lavoro è parallelizzabile e $1 - p$ no (qui: dividere l'intervallo, sommare i risultati, avviare i thread), con n core l'accelerazione è al massimo $1/((1 - p) + p/n)$, e per $n \rightarrow \infty$ tende a $1/(1 - p)$: la parte sequenziale, per piccola che sia, mette un tetto.

Perché 64 compiti e non uno per thread? Perché il lavoro non è uniforme: l'ultimo pezzo dell'intervallo costa più del primo, e con 8 compiti su 8 thread sette thread finiscono e aspettano l'ottavo (2,6 s invece di 2,0 s nella stessa misura). Compiti *più numerosi dei thread* si bilanciano da soli: chi finisce prende il successivo dalla coda. È la regola generale: tanti compiti piccoli, pochi thread.

Osservazione 4.1

`CompletableFuture` **JAVA 8** è un `Future` che si può *comporre*: `thenApply` trasforma il risultato quando arriva, `thenCombine` ne unisce due, `thenAccept` lo consuma, tutto senza chiamare `get()` e senza bloccare nessuno. È il modello su cui sono costruite le librerie di rete asincrone; in questa dispensa restiamo a `Future`, che basta per i casi di calcolo.

4.5 Concorrenza: quando i thread condividono

Finora i compiti erano indipendenti: ciascuno lavorava sui suoi dati e restituiva un risultato. Il momento in cui due thread leggono e scrivono *lo stesso oggetto* è quello in cui i programmi iniziano a dare risultati sbagliati, raramente, in modo non riproducibile.

4.5.1 Il contatore che perde

```
class Counter {
    int count = 0;
    void inc() { count++; }
}
```

```
ExecutorService executor = Executors.newFixedThreadPool(2);
Counter counter = new Counter();
for (int i = 0; i < 10_000; i++)
    executor.submit(counter::inc);
executor.shutdown();
executor.awaitTermination(1, TimeUnit.MINUTES);
System.out.println(counter.count);
```

9994

Diecimila incrementi, 9994 nel contatore; alla prova successiva 9991, poi 9997. Il programma è corretto quasi sempre, che è il modo peggiore di essere sbagliato. `count++` sembra un'istruzione ed è tre: leggi `count`, somma uno, scrivi `count`. Lo dice il bytecode:

```
$ javap -c Counter
void inc();
    0: aload_0
    1: dup
    2: getfield      #7      // leggi count
    5: iconst_1
    6: iadd          // somma 1
    7: putfield      #7      // scrivi count
   10: return
```

Fra la lettura e la scrittura lo scheduler può sospendere il thread (sezione 4.1: fra due istruzioni qualsiasi può passare un tempo arbitrario). Se i thread A e B leggono entrambi 41, entrambi sommano, entrambi scrivono 42: un incremento è perso.

Definizione 4.1: Conflitto e sezione critica

Si ha un *conflitto* (*race condition*) quando il risultato di un programma dipende dall'ordine, non controllato, in cui più thread accedono a uno stato condiviso. Il tratto di codice che accede allo stato condiviso e che deve essere eseguito da un thread alla volta per restare corretto è una *sezione critica*.

La soluzione, in tutte le sue forme, è sempre la stessa idea: rendere *atomica* la sezione critica, cioè far sì che nessun altro thread la veda a metà.

4.5.2 Visibilità: non è solo questione di atomicità

C'è un secondo problema, più sottile. Ogni core ha le sue cache, e il compilatore JIT ottimizza come se il thread fosse solo: una variabile letta in un ciclo può essere tenuta in un registro. Questo programma, su un portatile qualsiasi, non termina:

```
class Lavoro {
    boolean fermati = false;

    void esegui() {
        long k = 0;
        while (!fermati) k++;           // la JIT legge fermati una volta sola
        System.out.println("fermato dopo " + k);
    }
}
```

```
Lavoro lavoro = new Lavoro();
new Thread(lavoro::esegui).start();
pausa(500);
lavoro.fermati = true;                // main scrive, l'altro non vede
```

Il thread di lavoro non legge mai più `fermati` dalla memoria: nulla gli dice che un altro thread possa cambiarla. La parola *volatile* è esattamente quel segnale: una variabile *volatile* viene sempre letta e scritta in memoria, mai tenuta in un registro, e una scrittura è visibile a tutte le letture successive di qualsiasi thread. Con *volatile boolean fermati* il ciclo termina.

volatile però non rende atomico niente: `volatile int count` con `count++` perde incrementi esattamente come prima, perché leggi-somma-scrivi restano tre operazioni. La regola pratica: *volatile* per una *bandiera* che un thread scrive e altri leggono; per tutto ciò che si legge e si modifica insieme servono gli strumenti che seguono.

Osservazione 4.2

Il *Java Memory Model* descrive con precisione quando una scrittura di un thread è garantita visibile a un altro, tramite la relazione *happens-before*. I casi che servono nella pratica: tutto ciò che un thread fa prima di `start()` è visibile al thread avviato; tutto ciò che un thread fa prima di terminare è visibile a chi fa `join()` o `get()` su di lui; l'uscita da un blocco `synchronized` è visibile a chi entra poi nello stesso blocco; una scrittura *volatile* è visibile a chi legge dopo; `submit` su un `executor` è visibile al compito. Fuori da queste garanzie non c'è nessun ordine su cui contare.

4.5.3 synchronized

```
class Counter {  
    int count = 0;  
    synchronized void inc() { count++; }  
}
```

10000

Ogni oggetto Java possiede un *monitor* (o *lock intrinseco*): una serratura che un solo thread alla volta può tenere. Un metodo `synchronized` acquisisce il monitor dell'oggetto su cui è chiamato (`this`) all'ingresso e lo rilascia all'uscita, anche per eccezione; un thread che trova il monitor occupato passa in `BLOCKED` e aspetta. La forma a blocco permette di scegliere l'oggetto e di restringere la sezione critica al minimo:

```
void inc() {  
    long nPrimi = new PrimeTester(1, 1_000_000).call(); // calcolo lungo, fuori  
    synchronized (this) {  
        count += nPrimi; // accesso condiviso,  
        dentro  
    }  
}
```

Tre regole che discendono dal meccanismo.

- **Si protegge lo stato, non il codice.** Il monitor è dell'oggetto: due thread che chiamano `inc()` su due `Counter` diversi non si bloccano a vicenda, e due metodi diversi `synchronized` sullo stesso oggetto sì. *Tutti* gli accessi a un dato condiviso, anche le sole letture, devono passare dallo stesso monitor: una lettura non sincronizzata può vedere un valore vecchio (visibilità) o a metà (un `long` su hardware a 32 bit).
- **Il monitor è rientrante.** Un thread che tiene il monitor può entrare in altri metodi `synchronized` dello stesso oggetto, anche ricorsivamente, senza bloccarsi da solo. È ciò che permette a un metodo `synchronized` di chiamarne un altro dello stesso oggetto senza pensarci.
- **La sezione critica è corta.** Dentro il monitor ci sta ciò che tocca lo stato condiviso; il calcolo che non lo tocca sta fuori. Un `synchronized` attorno a un metodo lungo trasforma otto thread in uno, con in più i costi del lock.

Un metodo `static synchronized` usa il monitor dell'oggetto `Classe.class`: protegge i campi statici, ed è indipendente dai monitor delle istanze. Sincronizzare su `this` ha un difetto: chiunque abbia un riferimento all'oggetto può fare `synchronized (counter)` e interferire. Una classe che espone il proprio stato condiviso a molti fa meglio a usare un lock privato:

```
private final Object lock = new Object();  
  
void inc() {  
    synchronized (lock) { count++; }  
}
```

4.5.4 Coordinarsi: wait e notifyAll

Il monitor non serve solo a escludere: serve anche ad *aspettare* che lo stato diventi quello giusto. Un produttore mette elementi in un buffer di capienza fissa, un consumatore li prende; il produttore deve aspettare quando il buffer è pieno, il consumatore quando è vuoto.

```
class Buffer {
    private final Deque<String> coda = new ArrayDeque<>();
    private final int capienza;
    Buffer(int capienza) { this.capienza = capienza; }

    synchronized void metti(String s) throws InterruptedException {
        while (coda.size() == capienza) wait();    // pieno: rilascia e dormi
        coda.addLast(s);
        notifyAll();                               // forse qualcuno aspettava
    }

    synchronized String preni() throws InterruptedException {
        while (coda.isEmpty()) wait();
        String s = coda.removeFirst();
        notifyAll();
        return s;
    }
}
```

`wait()` si chiama tenendo il monitor: lo rilascia, mette il thread in `WAITING`, e quando il thread viene svegliato lo riacquisisce prima di ritornare. `notifyAll()` sveglia tutti i thread in attesa su quel monitor (`notify()` ne sveglia uno, a caso: quasi sempre è la scelta sbagliata). Il `while` attorno a `wait()` non è un `if` per due ragioni: fra la sveglia e il riacquisto del monitor un altro thread può aver cambiato di nuovo lo stato, e la JVM ammette risvegli spuri. Si ricontrolla sempre la condizione. Nella pratica questo schema è già scritto, e meglio, nelle code bloccanti della sezione 4.6; va conosciuto perché è ciò che sta sotto.

4.5.5 Deadlock

Due conti e un trasferimento che blocca entrambi, per non lasciare mai la somma a metà:

```
void trasferisci(Conto da, Conto a, int importo) {
    synchronized (da) {
        synchronized (a) {
            da.preleva(importo);
            a.versa(importo);
        }
    }
}
```

Due thread chiamano nello stesso istante `trasferisci(x, y, 10)` e `trasferisci(y, x, 20)`. Il primo prende il monitor di `x` e aspetta quello di `y`; il secondo ha preso `y` e aspetta `x`. Nessuno dei due cederà mai: è un *deadlock*, e il programma resta fermo senza errori, senza eccezioni, senza consumare processore. `jstack` lo riconosce e lo stampa (“Found one Java-level deadlock”). Si verifica quando quattro condizioni sono vere insieme: risorse esclusive, un thread che ne tiene una mentre ne chiede un’altra, nessuna prelazione, e un *ciclo* di attese. La

soluzione più semplice rompe l'ultima: tutti i thread acquisiscono i lock *nello stesso ordine*, per esempio quello di un identificatore dei conti.

```
void trasferisci(Conto da, Conto a, int importo) {
    Conto primo = da.id() < a.id() ? da : a;
    Conto secondo = da.id() < a.id() ? a : da;
    synchronized (primo) {
        synchronized (secondo) {
            da.preleva(importo);
            a.versa(importo);
        }
    }
}
```

L'altra via è non aspettare per sempre: i lock espliciti della sezione seguente hanno `tryLock(t, unit)`, che rinuncia dopo un tempo e permette di rilasciare quello che si aveva e riprovare.

Osservazione 4.3

Due parenti del deadlock. *Starvation*: un thread che non ottiene mai la risorsa perché altri, più fortunati o più prioritari, la prendono sempre prima. *Livelock*: due thread che, per evitarsi, cambiano stato in continuazione senza avanzare, come due persone in un corridoio che si spostano sempre dallo stesso lato. Entrambi si vedono come un programma che gira ma non conclude.

4.5.6 Thread safety per costruzione

Il modo migliore di proteggere uno stato condiviso è non averne. Tre strategie, dalla più semplice:

- **Immutabilità.** Un oggetto che non cambia dopo la costruzione può essere letto da qualsiasi numero di thread senza nessun lock: `String`, gli involucri dei primitivi, `List.of(...)`, i record con campi immutabili. Un “cambiamento” produce un oggetto nuovo.
- **Confinamento.** Un dato usato da un solo thread non è condiviso: le variabili locali lo sono per natura (ogni thread ha il suo stack), e un oggetto creato dal compito e mai passato ad altri lo è di fatto. È il modello di `PrimeTester`: ogni compito conta nel suo campo, e la condivisione avviene una volta sola, attraverso il `Future`.
- **Oggetti thread-safe dichiarati.** Le classi della libreria che lo sono lo dicono nella documentazione; quelle che non lo dicono, non lo sono. Non lo sono, per esempio, `ArrayList`, `HashMap` e `StringBuilder`; lo sono `ConcurrentHashMap`, `AtomicInteger` e le code del package `java.util.concurrent`.

4.6 Gli strumenti di `java.util.concurrent`

`synchronized` è il meccanismo di base; la libreria ne offre versioni più flessibili e, per i casi comuni, soluzioni già pronte che evitano di scrivere lock.

4.6.1 Lock espliciti: `ReentrantLock`

```
private final ReentrantLock lock = new ReentrantLock();
private int count = 0;

void inc() {
    lock.lock();
    try {
        count++;
    } finally {
        lock.unlock();           // sempre, anche in caso di eccezione
    }
}
```

Stesso comportamento di `synchronized`, con il rilascio a carico del programmatore (da qui il `finally`, obbligatorio) e qualche possibilità in più: `tryLock()` prova senza bloccarsi e dice se ha avuto il lock, `tryLock(t, unit)` aspetta al massimo un tempo, `lockInterruptibly()` si lascia interrompere, il costruttore `new ReentrantLock(true)` rende l'ordine di accesso equo (chi aspetta da più tempo entra prima), e `isHeldByCurrentThread()` aiuta a diagnosticare. Si usa `ReentrantLock` quando serve una di queste cose; altrimenti `synchronized` è più corto e altrettanto veloce.

```
ExecutorService executor = Executors.newFixedThreadPool(2);
ReentrantLock lock = new ReentrantLock();

executor.submit(() -> {
    lock.lock();
    try { pausa(1000); } finally { lock.unlock(); }
});
executor.submit(() -> {
    System.out.println("occupato: " + lock.isLocked());           // true
    System.out.println("da me: " + lock.isHeldByCurrentThread()); // false
    System.out.println("preso: " + lock.tryLock());              // false
});
```

4.6.2 Lettori e scrittori: `ReadWriteLock`

Molti thread possono *leggere* una struttura insieme senza danno; il problema nasce solo quando qualcuno *scrive*. `ReentrantReadWriteLock` ha due lock coordinati: quello di lettura, che più thread possono tenere insieme, e quello di scrittura, esclusivo e incompatibile con le letture.

```
Map<String, String> mappa = new HashMap<>();
ReadWriteLock lock = new ReentrantReadWriteLock();

executor.submit(() -> {
    lock.writeLock().lock();
    try {
        pausa(1000);
        mappa.put("alfa", "primo elemento");
    } finally { lock.writeLock().unlock(); }
});

Runnable lettura = () -> {
    lock.readLock().lock();
    try {
        System.out.println(mappa.get("alfa"));
        pausa(1000);
    } finally { lock.readLock().unlock(); }
};
executor.submit(lettura);
executor.submit(lettura);           // le due letture girano insieme, dopo la
                                     scrittura
```

StampedLock **JAVA 8** è una variante più veloce con una modalità in più, la *lettura ottimistica*: si legge senza prendere nessun lock, e `validate(stamp)` dice a posteriori se nel frattempo c'è stata una scrittura (nel qual caso si rilegge con il lock vero). Non è rientrante e richiede di passare lo `stamp` restituito da ogni acquisizione al rilascio: va riservato ai casi in cui le misure dicono che `ReadWriteLock` è il collo di bottiglia.

4.6.3 Risorse a posti limitati: Semaphore

Un lock ammette un thread; un semaforo ne ammette *n*. Nasce con un numero di *permessi*; `acquire()` ne prende uno o aspetta, `release()` lo restituisce. Il caso tipico è limitare l'accesso a una risorsa che regge solo un certo numero di utenti simultanei: connessioni, licenze, memoria.

```

ExecutorService executor = Executors.newFixedThreadPool(10);
Semaphore semaforo = new Semaphore(5);

Runnable compito = () -> {
    boolean permesso = false;
    try {
        permesso = semaforo.tryAcquire(1, TimeUnit.SECONDS);
        if (permesso) {
            System.out.println("dentro: " + Thread.currentThread().getName());
            pausa(5000);
        } else {
            System.out.println("rinuncio: " + Thread.currentThread().getName());
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    } finally {
        if (permesso) semaforo.release();
    }
};
for (int i = 0; i < 10; i++) executor.submit(compito);

```

Cinque thread entrano, cinque aspettano un secondo e rinunciano. Da notare il `finally` che rilascia *solo* se il permesso era stato ottenuto: rilasciare senza aver acquisito aumenta i permessi, e il semaforo da quel momento ammette un thread in più.

4.6.4 Variabili atomiche

Per un contatore, una bandiera, un riferimento da aggiornare in modo atomico, il lock è sproporzionato. Le classi di `java.util.concurrent.atomic` (`AtomicInteger`, `AtomicLong`, `AtomicBoolean`, `AtomicReference<V>`) offrono operazioni atomiche senza lock, implementate con un'istruzione del processore, il *compare-and-swap*: “scrivi il nuovo valore solo se quello attuale è ancora quello che ho letto, altrimenti dimmelo”. Se il confronto fallisce la libreria rilegge e riprova; non c'è mai un thread bloccato.

```

AtomicInteger count = new AtomicInteger(0);
for (int i = 0; i < 10_000; i++)
    executor.submit(count::incrementAndGet);
...
System.out.println(count.get()); // 10000

```

```

count.updateAndGet(n -> n + 2); // qualsiasi funzione,
    atomica
count.accumulateAndGet(5, Math::max);
boolean fatto = count.compareAndSet(42, 0); // se vale 42, azzera

```

`updateAndGet` riceve una lambda che può essere eseguita *più volte*, se nel frattempo un altro thread ha cambiato il valore: deve essere pura. Quando tanti thread incrementano *lo stesso* contatore in continuazione (il numero di richieste di un server) il *compare-and-swap* fallisce spesso e si riprova spesso: per questo caso esistono `LongAdder` e `LongAccumulator` **JAVA 8**, che tengono una cella per thread e sommano solo quando si chiede `sum()`. Costano più memoria, e la lettura è meno precisa durante gli aggiornamenti; in cambio scalano.

Le prestazioni dipendono dall'hardware più di quanto si creda, e la sola regola seria è misurare. Dieci milioni di incrementi da 16 thread sullo stesso portatile della sezione 4.4.4: `synchronized` 74 ms, `AtomicInteger` 391 ms, `LongAdder` 66 ms. Il compare-and-swap, sotto contesa altissima, ha perso dal lock; su un altro processore il risultato può rovesciarsi. L'atomica resta la scelta giusta per il caso comune, la contesa moderata, dove non blocca nessuno e non costa una chiamata al sistema.

4.6.5 Collezioni concorrenti

Le collezioni di `java.util` non sono thread-safe. `Collections.synchronizedList(list)`, la scorciatoia storica, avvolge ogni metodo in un `synchronized` e ha un limite che sorprende: l'iterazione non è protetta, perché è fatta di molte chiamate separate, e un `for` su una lista che un altro thread sta modificando finisce con l'eccezione `ConcurrentModificationException`. Le classi del package `java.util.concurrent` sono invece progettate per la concorrenza dall'interno.

```
ConcurrentMap<String, Integer> conteggi = new ConcurrentHashMap<>();
conteggi.put("alfa", 1);
conteggi.putIfAbsent("alfa", 99); // resta 1
conteggi.merge("alfa", 1, Integer::sum); // 2, atomico
conteggi.compute("beta", (k, v) -> v == null ? 1 : v + 1);
conteggi.forEach((k, v) -> System.out.println(k + " = " + v));
```

`ConcurrentHashMap` suddivide la mappa in segmenti con lock indipendenti, così scritture su chiavi diverse non si bloccano a vicenda, e le letture non prendono lock. I metodi `merge`, `compute`, `computeIfAbsent` sono atomici: la coppia “leggi, poi scrivi” che con una `HashMap` e un lock esterno sarebbe una sezione critica qui è una chiamata sola. In più ha operazioni che la libreria esegue in parallelo quando la mappa supera una soglia di elementi:

```
ConcurrentHashMap<String, Integer> mappa = new ConcurrentHashMap<>();
...
mappa.forEach(1, (k, v) -> // soglia 1: sempre
    parallelo
        System.out.printf("%s=%d [%s]%n", k, v,
            Thread.currentThread().getName()));
int totale = mappa.reduceValues(1, Integer::sum);
```

`CopyOnWriteArrayList` è una lista che a ogni modifica copia l'array interno: le letture e le iterazioni non prendono mai lock e vedono un'istantanea coerente, le scritture costano una copia. È la struttura giusta per un elenco di *listener*: pochi cambiamenti, molte scansioni.

`BlockingQueue` è il produttore/consumatore della sezione 4.5 già scritto: `put` aspetta se la coda è piena, `take` se è vuota, con le varianti con `timeout` `offer` e `poll`. `ArrayBlockingQueue` ha capienza fissa, `LinkedBlockingQueue` opzionale, `PriorityBlockingQueue` estrae per priorità.

```

BlockingQueue<String> coda = new ArrayBlockingQueue<>(10);

executor.submit(() -> {
    for (int i = 0; i < 100; i++) coda.put("msg " + i);
    coda.put("FINE");
    return null;
});
executor.submit(() -> {
    String s;
    while (!(s = coda.take()).equals("FINE"))
        System.out.println(s);
    return null;
});

```

Le due lambda terminano con `return null`: così sono `Callable` e possono lanciare l'eccezione di `put` e `take` senza un `try`, un piccolo trucco che vale la pena conoscere. È lo schema con cui un thread di rete consegna i messaggi ricevuti a chi li elabora, e con cui l'interfaccia grafica, come vedremo, riceve il lavoro da fare.

4.6.6 Quale strumento

Il problema	Lo strumento
una bandiera scritta da uno, letta da altri	<code>volatile boolean</code> o <code>AtomicBoolean</code>
un contatore, un riferimento da aggiornare	<code>AtomicInteger/Long/Reference</code> ; <code>LongAdder</code> se la contesa è alta
più campi da aggiornare insieme	<code>synchronized</code> su un lock privato; <code>ReentrantLock</code> se servono timeout o equità
molte letture, poche scritture	<code>ReadWriteLock</code> ; <code>CopyOnWriteArrayList</code> per gli elenchi
una risorsa a <i>n</i> posti	<code>Semaphore</code>
passare dati fra thread	<code>BlockingQueue</code>
una mappa condivisa	<code>ConcurrentHashMap</code> con <code>merge/compute</code>
aspettare che una condizione diventi vera	<code>wait/notifyAll</code> , <code>Condition</code> di <code>ReentrantLock</code> , <code>CountDownLatch</code>
nessuno stato condiviso	la scelta migliore: immutabilità e confinamento

4.7 Esecuzione differita e periodica

`ScheduledExecutorService`, creato con `Executors.newScheduledThreadPool(n)`, aggiunge il tempo: un compito può partire dopo un ritardo, oppure ripetersi.

```

ScheduledExecutorService executor = Executors.newScheduledThreadPool(1);
Runnable task = () -> System.out.println("eseguito alle " + LocalTime.now());

ScheduledFuture<?> future = executor.schedule(task, 3, TimeUnit.SECONDS);
pausa(1400);
System.out.printf("mancano %d ms%n", future.getDelay(TimeUnit.MILLISECONDS));

```

```
mancano 1598 ms  
eseguito alle 10:42:17.003
```

Per i compiti periodici ci sono due metodi, e la differenza conta quando il compito è lento.

```
executor.scheduleAtFixedRate(task, 0, 1, TimeUnit.SECONDS);
```

`scheduleAtFixedRate` avvia il compito ogni secondo, *contato dall'inizio dell'esecuzione precedente*: se il compito dura tre secondi, le esecuzioni si susseguono senza pausa (0, 3, 6, 9: ogni inizio alla prima scadenza utile; non si accumulano esecuzioni arretrate, e con un solo thread non si sovrappongono).

```
executor.scheduleWithFixedDelay(task, 0, 1, TimeUnit.SECONDS);
```

`scheduleWithFixedDelay` aspetta un secondo *dalla fine* dell'esecuzione precedente: con lo stesso compito da tre secondi, gli inizi sono a 0, 4, 8, 12. Il primo serve a un orologio o a un campionamento a frequenza fissa; il secondo a un lavoro di manutenzione che deve lasciare respiro fra una corsa e l'altra.

Attenzione 4.3

Un'eccezione non catturata in un compito periodico *ferma le ripetizioni*, in silenzio: il `ScheduledFuture` la conserva, ma nessuno di solito lo interroga, e il compito smette di girare senza che appaia niente. Un compito periodico avvolge il suo corpo in un `try/catch` che registra l'errore e lascia continuare le ripetizioni.

4.8 Thread e stream paralleli

Con quanto visto si può chiudere un discorso lasciato aperto nella dispensa sugli stream: che cosa succede con `parallelStream()`.

```
long n = LongStream.rangeClosed(1, 100_000_000)  
    .parallel()  
    .filter(PrimeTester::isPrime)  
    .count();
```

La libreria spacca la sorgente in pezzi (con `trySplit`, finché i pezzi sono abbastanza piccoli), consegna ogni pezzo come compito al `ForkJoinPool` comune della JVM, un pool a *work stealing* con tanti thread quanti i core meno uno (il chiamante partecipa), e fonde i risultati parziali. In pratica è il programma della sezione 4.4.4 scritto in quattro righe, con lo stesso tempo di esecuzione, e vale per questa versione tutto ciò che valeva per quella: il lavoro dev'essere puro calcolo su elementi indipendenti, le lambda non devono toccare stato condiviso (girano su thread diversi, in un ordine che dipende dalla divisione) e il guadagno c'è solo se gli elementi sono molti e il lavoro per elemento non è banale.

Due cose in più, ora che sappiamo che cosa c'è sotto. Il pool comune è *uno per JVM*: tutti gli stream paralleli del programma, e i `CompletableFuture` senza executor esplicito, condividono gli stessi thread. Un compito che vi *aspetta* (una lettura di rete dentro una lambda) tiene fermo un thread del pool, e con pochi compiti così tutto il parallelismo della JVM si ferma: gli stream paralleli sono per il calcolo, non per l'I/O. E non è per i casi in cui serve controllare i thread: chi vuole un numero diverso di thread, o isolare un lavoro dagli altri,

usa `Executors.newWorkStealingPool(n)`, che è un `ForkJoinPool` privato, e vi sottomette i compiti a mano.

4.9 Thread e Swing: la coda degli eventi

Tutto ciò che precede si incontra, in forma concentrata, nel primo programma con un'interfaccia grafica che fa un calcolo lungo. Questa sezione dà per note le basi di Swing (una finestra, dei componenti, un *listener* che reagisce a un pulsante) e spiega che cosa c'è sotto, perché senza saperlo è impossibile scrivere un'interfaccia che resti viva.

4.9.1 Un'interfaccia è un ciclo

Un programma a eventi non “fa” qualcosa dall'inizio alla fine: aspetta. Da qualche parte c'è un ciclo che riceve gli eventi (un click, un tasto, una finestra da ridisegnare) e per ciascuno chiama il codice registrato. In Swing quel ciclo gira su un thread dedicato, l'*Event Dispatch Thread* (EDT), che nel debugger si chiama `AWT-EventQueue-0`. È lui che esegue ogni listener, e ogni `paintComponent`, e il `Runnable` passato a `invokeLater`. E vale una regola, scritta nella documentazione di ogni componente:

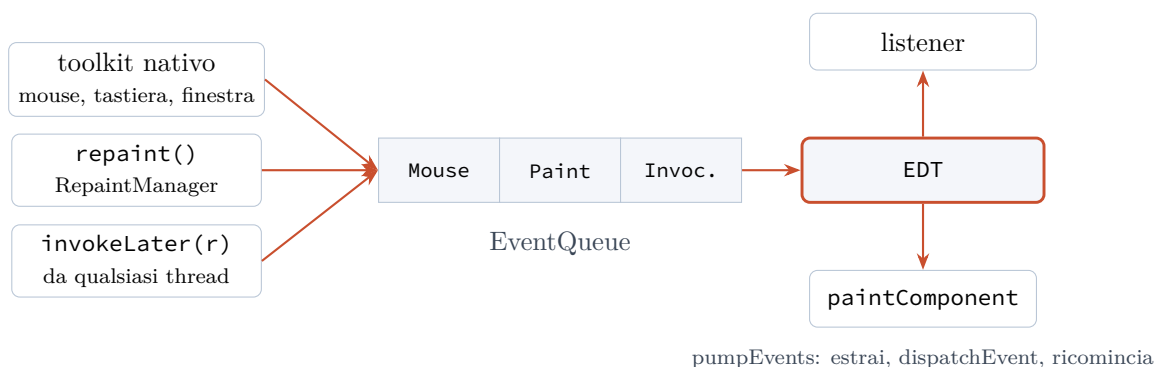
Definizione 4.2: La regola del singolo thread di Swing

I componenti Swing non sono thread-safe. Ogni accesso a un componente, dalla creazione alla lettura di una proprietà alla modifica, va eseguito dall'Event Dispatch Thread.

Perché una libreria moderna rinuncia alla thread safety? Perché un'interfaccia è un albero di componenti che si modificano a vicenda in continuazione (un testo che cambia fa ridisegnare il contenitore, che ricalcola il layout, che sposta i fratelli): proteggere ogni componente con un lock produrrebbe deadlock a ogni angolo, come i due conti della sezione 4.5, e i toolkit che ci hanno provato sono tornati indietro. Un solo thread è la scelta di tutte le librerie grafiche di oggi, Swing, JavaFX, Android, il web. Il prezzo è che quel thread *non va mai bloccato*.

4.9.2 Come funziona EventQueue

Il cuore è la classe `java.awt.EventQueue`: una coda di oggetti `AWTEvent`, con un produttore e un consumatore.



- **I produttori.** Il *toolkit nativo* ha un thread suo (`AppKit Thread` su macOS, `AWT-XAWT` su Linux, `AWT-Windows`) che riceve gli eventi dal sistema operativo e li traduce in `MouseEvent`, `KeyEvent`, `WindowEvent`, accodandoli con `postEvent`. Il programma stesso è un produttore in due modi: `repaint()` non disegna niente, chiede al `RepaintManager` di accodare un

`PaintEvent` (e più richieste vicine vengono *fuse* in una sola: cento `repaint` in un ciclo costano un disegno); e `EventQueue.invokeLater(r)` incarta un `Runnable` in un `InvocationEvent` e lo accoda. È l'unico modo con cui un altro thread può far eseguire qualcosa all'EDT.

- **Il consumatore.** L'EDT esegue il ciclo `pumpEvents`: estrae l'evento in testa, chiama `dispatchEvent`, che lo recapita al componente giusto (`Component.dispatchEvent`, `processEvent`), che chiama i suoi listener uno dopo l'altro; per un `PaintEvent` chiama `paint`, per un `InvocationEvent` chiama `run()`. Finito, ricomincia. Un solo thread, un evento alla volta, nell'ordine della coda.

Da questo meccanismo discende tutto ciò che conta. `revalidate()` e `repaint()` sono *richieste*, soddisfatte quando l'EDT arriverà all'evento, non azioni immediate: chiamarle dentro un listener non disegna niente finché il listener non ritorna. Due `invokeLater` dallo stesso thread vengono eseguiti nell'ordine in cui sono stati accodati. E un listener che non ritorna ferma *tutta* la coda.

4.9.3 Chi avvia che cosa

```
public class PrimeFinderApp extends JFrame {
    private final JTextField campoMax = new JTextField("100000000", 10);
    private final JButton btnConta = new JButton("Conta");
    private final JProgressBar barra = new JProgressBar(0, 100);
    private final JLabel lblRisultato = new JLabel("-", SwingConstants.CENTER);

    public PrimeFinderApp() {
        super("Numeri primi");
        JPanel sopra = new JPanel();
        sopra.add(new JLabel("Fino a:"));
        sopra.add(campoMax);
        sopra.add(btnConta);
        add(sopra, BorderLayout.NORTH);
        add(barra, BorderLayout.CENTER);
        add(lblRisultato, BorderLayout.SOUTH);
        btnConta.addActionListener(e -> conta());
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        pack();
    }

    public static void main(String[] args) {
        EventQueue.invokeLater(() -> new PrimeFinderApp().setVisible(true));
    }
}
```

Il `main` gira sul thread `main`, che per la regola della definizione 4.2 non deve toccare i componenti: per questo non costruisce la finestra, ma accoda la costruzione con `invokeLater` e termina subito. Il programma non termina con lui: l'EDT e il thread del toolkit non sono *daemon* e restano vivi finché esiste una finestra; `EXIT_ON_CLOSE` chiude la JVM alla chiusura dell'ultima. `EventQueue.invokeLater` e `SwingUtilities.invokeLater` sono lo stesso metodo; `EventQueue.isDispatchThread()` dice se il thread corrente è l'EDT, e serve per scrivere codice che funziona chiamato da entrambe le parti.

4.9.4 Il blocco

La prima versione di `conta()` è quella che tutti scrivono:

```
private void conta() {                                // sull'EDT: congela la finestra
    long max = Long.parseLong(campoMax.getText());
    lblRisultato.setText("conto...");
    long n = new PrimeTester(1, max).call();           // 14 secondi
    lblRisultato.setText("primi: " + n);
}
```

Al click la finestra si blocca per quattordici secondi: non si ridisegna, non risponde al mouse, il sistema la segnala come “non risponde”; l’etichetta non mostra nemmeno “conto...”. Il perché è tutto nel ciclo: `conta` è il listener del pulsante, l’EDT lo sta eseguendo, e finché non ritorna non estrae nessun altro evento, nemmeno il `PaintEvent` accodato dal `setText`. Il risultato arriva, ma dopo che tutto si è fermato.

4.9.5 Lavoro fuori, rientro dentro

Il calcolo va su un altro thread; ma il risultato va mostrato su un componente, e toccarlo da quel thread viola la regola. La soluzione è in due tempi: il lavoro fuori dall’EDT, l’aggiornamento rientra sull’EDT con `invokeLater`.

```
private void conta() {
    long max = Long.parseLong(campoMax.getText()); // 1. leggo sull'EDT
    btnConta.setEnabled(false);
    lblRisultato.setText("conto...");
    new Thread(() -> {                               // 2. calcolo su un altro
        thread
        long n = new PrimeTester(1, max).call();
        EventQueue.invokeLater(() -> {                // 3. rientro sull'EDT
            lblRisultato.setText("primi: " + n);
            btnConta.setEnabled(true);
        });
    }).start();
}
```

Il listener ritorna in un istante e l’EDT torna a svuotare la coda: la finestra si ridisegna, il pulsante appare disabilitato, l’etichetta dice “conto...”. Quattordici secondi dopo il thread di lavoro accoda un `InvocationEvent`, e l’EDT, al suo turno, aggiorna l’etichetta. Si noti che `max` viene letto *prima*, sull’EDT: leggere `campoMax.getText()` dentro il thread di lavoro sarebbe un accesso a un componente dal thread sbagliato.

Che cosa succede se non si rientra, e si chiama `setText` direttamente dal thread di lavoro? Quasi sempre funziona, ed è il problema. Non c’è nessun controllo e nessuna eccezione: il componente viene modificato mentre l’EDT, magari, lo sta disegnando o ne sta ricalcolando il layout, e il risultato è un’interfaccia che una volta ogni mille mostra un valore vecchio, un testo tagliato, un componente al posto sbagliato, o lancia un’eccezione dal profondo di Swing che non si riesce a riprodurre. Un programma del genere, con un’etichetta aggiornata cento volte al secondo da un thread, prima o poi lo fa vedere.

`invokeAndWait(r)` è la variante che aspetta l’esecuzione di `r` prima di ritornare: serve quando il thread di lavoro ha bisogno di una risposta dall’interfaccia (un valore da un campo, una conferma). Chiamato *dall’EDT* lancia un `Error`, perché sarebbe un deadlock certo: l’EDT

aspetterebbe un evento che solo lui può eseguire. In pratica si evita, e si legge tutto ciò che serve prima di avviare il lavoro.

Ricetta: eseguire sull'EDT da qualunque thread

```
void aggiorna(String testo) {  
    Runnable r = () -> lblRisultato.setText(testo);  
    if (EventQueue.isDispatchThread())  
        r.run();                               // sono già sull'EDT: subito  
    else  
        EventQueue.invokeLater(r);             // altrimenti: in coda  
}
```

Un metodo del genere può essere chiamato da un listener e da un thread di lavoro senza che chi lo chiama debba sapere dove si trova.

4.9.6 SwingWorker

Lo schema “lavoro fuori, rientro dentro” è così comune che Swing lo fornisce pronto, con in più l'avanzamento, i risultati parziali e l'annullamento: `SwingWorker<T, V>`, dove `T` è il tipo del risultato finale e `V` quello dei risultati intermedi.

```

private void conta() {
    long max = Long.parseLong(campoMax.getText());
    btnConta.setEnabled(false);
    SwingWorker<Long, Void> worker = new SwingWorker<>() {
        @Override
        protected Long doInBackground() {           // su un thread di lavoro
            int nBlocchi = 100;
            long perBlocco = max / nBlocchi, totale = 0;
            for (int b = 0; b < nBlocchi && !isCancelled(); b++) {
                totale += new PrimeTester(
                    b * perBlocco + 1, (b + 1) * perBlocco).call();
                setProgress(b + 1);                 // 0..100
            }
            return totale;
        }

        @Override
        protected void done() {                     // sull'EDT, alla fine
            try {
                lblRisultato.setText("primi: " + get());
            } catch (CancellationException e) {
                lblRisultato.setText("annullato");
            } catch (InterruptedException | ExecutionException e) {
                lblRisultato.setText("errore: " + e.getCause());
            }
            btnConta.setEnabled(true);
        }
    };
    worker.addPropertyChangeListener(e -> {        // sull'EDT
        if ("progress".equals(e.getPropertyName()))
            barra.setValue((Integer) e.getNewValue());
    });
    worker.execute();
}

```

I pezzi, e dove girano:

- `doInBackground()` è il compito: gira su un thread di un pool interno a Swing, e non deve toccare componenti. Restituisce il risultato o lancia.
- `done()` viene chiamato sull'EDT quando il compito è finito, in qualsiasi modo. `get()` qui non blocca: il risultato c'è già, oppure c'è un'eccezione da rilanciare (`ExecutionException` per un errore nel compito, `CancellationException` se è stato annullato).
- `setProgress(0..100)` dal compito aggiorna la proprietà `progress`; chi si è registrato con `addPropertyChangeListener` viene avvisato sull'EDT. È il canale per la barra di avanzamento.
- `publish(v...)` dal compito e `process(List<V>)` sull'EDT sono il canale per i risultati intermedi (le righe di un file man mano che si leggono): Swing raggruppa i valori pubblicati e li consegna a blocchi, così mille `publish` non sono mille eventi.

- `cancel(true)` interrompe il thread del compito e fa valere `isCancelled()`: il ciclo di `doInBackground` lo controlla a ogni blocco e si ferma. È l'interruzione cooperativa della sezione 4.2, con un nome nuovo.
- `execute()` accoda il worker; un `SwingWorker` si esegue una volta sola, per un nuovo calcolo se ne crea uno nuovo.

4.9.7 javax.swing.Timer

Un orologio, un'animazione, un controllo periodico dello stato: serve un compito ripetuto che tocca i componenti. `ScheduledExecutorService` lo eseguirebbe sul thread sbagliato. `javax.swing.Timer` è un timer che consegna i suoi `ActionEvent` *sull'EDT*, attraverso la stessa coda di tutto il resto:

```
Timer timer = new Timer(1000, e -> // ogni secondo, sull'EDT
    lblOra.setText(LocalTime.now().toString()));
timer.start();
...
timer.stop();
```

Il listener può toccare i componenti senza nessun rientro; in cambio dev'essere breve, perché gira sull'EDT. La regola per scegliere: `javax.swing.Timer` per compiti brevi che aggiornano l'interfaccia, `ScheduledExecutorService` per il lavoro vero, con il rientro via `invokeLater` quando c'è qualcosa da mostrare. (`java.util.Timer` è una terza classe, più vecchia, con un thread proprio e senza nessun legame con l'EDT: con Swing non si usa.)

Ricetta: thread e Swing

1. **La finestra nasce sull'EDT:** main fa `EventQueue.invokeLater(() -> new App().setVisible(true))` e basta.
2. **L'EDT non si blocca mai:** niente calcoli lunghi, niente `sleep`, niente rete nei listener. Un listener dura millisecondi.
3. **Il lavoro va altrove:** un `SwingWorker` per un'operazione con risultato, un executor per il lavoro continuo, `javax.swing.Timer` per il periodico breve.
4. **Si rientra sempre:** ogni accesso a un componente da un altro thread passa da `invokeLater`, `done()`, `process()` o un listener di proprietà.
5. **Il modello condiviso è protetto o confinato:** i dati che il thread di lavoro produce e l'interfaccia mostra sono immutabili, oppure passano in una coda, oppure sono protetti da un lock; e si leggono dall'interfaccia prima di partire, non durante.

4.10 Thread virtuali

JAVA 21 Chiudiamo il discorso aperto nella sezione 4.1. I thread di piattaforma sono uno a uno con quelli del sistema operativo, e questo mette un tetto: un server che dedica un thread a ogni connessione, con diecimila connessioni aperte che per lo più *aspettano* dati, tiene diecimila stack e diecimila thread del kernel per fare, in ogni istante, pochissimo. Il pool limita il numero di thread, ma allora le connessioni in più aspettano in coda.

Java 21 aggiunge i *thread virtuali*: thread gestiti dalla JVM e non dal kernel. Sono oggetti Thread a tutti gli effetti, con lo stesso codice, ma leggeri (lo stack vive nell'heap e cresce quanto serve) e numerosi (milioni, non migliaia). La JVM li esegue *montandoli* su un piccolo pool di thread di piattaforma, i *carrier*; quando un thread virtuale si blocca (*sleep*, una lettura di rete, una coda, un lock della libreria) viene *smontato*, il carrier passa a un altro, e al risveglio viene rimontato dove c'è posto. È un modello *molti a molti*, ed è il punto in cui l'astrazione di Java smette di coincidere con quella del sistema operativo.

```
Thread t = Thread.ofVirtual().start(() -> System.out.println("virtuale"));
t.join();

try (ExecutorService executor = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 10_000; i++)
        executor.submit(() -> { pausa(1000); return null; });
}                                     // close(): aspetta tutti
```

Diecimila compiti che dormono un secondo: con un pool di 200 thread di piattaforma ci vogliono 50 secondi (cinquanta turni da 200), con un thread virtuale per compito poco più di un secondo. Non è magia: i compiti *aspettano*, e aspettare in diecimila non costa niente se nessuno occupa un thread del kernel per farlo.

Le regole d'uso, che derivano dal meccanismo:

- **Per l'attesa, non per il calcolo.** Un thread virtuale che calcola occupa il suo carrier esattamente come un thread di piattaforma: per i numeri primi non cambia niente. Il guadagno c'è con molti compiti bloccati su I/O.
- **Non si mettono in pool.** Costano così poco che si crea un thread per compito e lo si butta: `newVirtualThreadPerTaskExecutor` fa esattamente questo, e un pool di thread virtuali è un controsenso.
- **Attenzione a `synchronized` (in Java 21).** Un thread virtuale bloccato *dentro* un blocco `synchronized` resta *inchiodato* al carrier, che non può servire altri: in Java 21 nelle sezioni critiche lunghe dei thread virtuali conviene `ReentrantLock`. Le versioni successive hanno tolto il limite.
- **Tutto il resto è uguale.** Lock, code, variabili atomiche, executor, interruzione: il codice scritto per i thread di piattaforma gira sui virtuali senza modifiche, ed è la ragione per cui vale la pena averli imparati prima.

4.11 Riepilogo

- Un **thread** è un flusso di esecuzione con stack e registri propri dentro un processo, di cui condivide l'heap; il sistema operativo lo schedula a fette di tempo e può sospenderlo fra due istruzioni qualsiasi. Il Thread di Java è uno a uno con un thread del kernel; la JVM termina con l'ultimo thread non daemon.
- Il compito è un **Runnable** (o un **Callable** se restituisce); `start()` lo avvia su un thread nuovo, `run()` no. Un thread si ferma solo se collabora: `interrupt()` alza una bandiera che le attese trasformano in `InterruptedException`, da non ignorare mai.
- Un **executor** separa i compiti dai thread: pochi thread, una coda, `submit` che ritorna un `Future`. Si spegne con `shutdown` e `awaitTermination`. Per il calcolo tanti thread quanti i core, e compiti più numerosi dei thread.
- Due thread che modificano lo stesso dato producono un **conflitto**: `count++` è tre istruzioni. La sezione critica si protegge con `synchronized` (il monitor dell'oggetto, rientrante), e la visibilità di una bandiera con `volatile`. Lock presi in ordine diverso danno **deadlock**: un ordine fisso lo evita.
- `java.util.concurrent` fornisce `ReentrantLock` e `ReadWriteLock`, `Semaphore`, le **variabili atomiche** e `LongAdder`, `ConcurrentHashMap` e `BlockingQueue`. La scelta migliore è non condividere: immutabilità e confinamento.
- `ScheduledExecutorService` esegue compiti ritardati o periodici; un'eccezione ferma le ripetizioni. Gli stream paralleli usano il `ForkJoinPool` comune: solo calcolo puro.
- **Swing** ha un solo thread, l'EDT, che svuota la `EventQueue` un evento alla volta: ogni accesso ai componenti avviene lì, e un listener lento congela tutto. Il lavoro va fuori (`SwingWorker`, un executor) e l'aggiornamento rientra con `invokeLater`; `javax.swing.Timer` per i compiti periodici brevi.
- I **thread virtuali** di Java 21 sono montati e smontati su pochi thread di piattaforma: a milioni, per compiti che aspettano, con lo stesso codice.

4.12 Esercizi

Esercizio 4.1: Prevedere l'ordine

Senza eseguirlo, dire che cosa stampa il programma e quali righe potrebbero scambiarsi di posto da un'esecuzione all'altra. Poi modificarlo, usando `join`, in modo che l'ordine sia sempre A, B, fine.

```
Thread a = new Thread(() -> { pausa(50); System.out.println("A"); });
Thread b = new Thread(() -> { pausa(10); System.out.println("B"); });
a.start(); b.start();
System.out.println("fine");
```

Esercizio 4.2: Il contatore che perde

Correggere Counter della sezione 4.5 in tre modi: con `synchronized`, con `AtomicInteger`, con `LongAdder`. Per ciascuno dire che cosa cambia nell'interfaccia della classe (il tipo restituito da `get()`) e in quale situazione lo si sceglierebbe.

Esercizio 4.3: Misurare i primi

Eseguire il programma della sezione 4.4.4 con 1, 2, 4, 8 e 16 thread sulla propria macchina, e con `nTask` pari a `nThread` e poi a 64. Riportare i tempi in una tabella e spiegare: dove smette di crescere l'accelerazione, e perché; perché con `nTask = nThread` è più lento.

Esercizio 4.4: Deadlock

Scrivere una classe Conto con `id`, `saldo`, `preleva` e `versa`, e la prima versione di `trasferisci` della sezione 4.5. Avviare due thread che trasferiscono in continuazione da `x` a `y` e da `y` a `x`, e osservare il blocco (con `jstack` o il debugger). Poi risolverlo con l'ordine di acquisizione e verificare che la somma dei saldi resti costante.

Esercizio 4.5: Produttore e consumatore

Un produttore genera i numeri da 1 a 1000 e un consumatore ne stampa la somma finale. Realizzarlo prima con il Buffer a `wait/notifyAll` della sezione 4.5, poi con una `ArrayBlockingQueue<Integer>` di capienza 10. Come fa il consumatore a sapere che non arriverà più niente?

Esercizio 4.6: Periodico e lento

Con un `ScheduledExecutorService` a un thread, avviare un compito che stampa l'ora e dorme tre secondi, con periodo di un secondo, prima con `scheduleAtFixedRate` e poi con `scheduleWithFixedDelay`. Prevedere gli orari, poi verificarli. Che cosa succede, nel primo caso, se il pool ha due thread?

Esercizio 4.7: La finestra che congela

Scrivere `PrimeFinderApp` nella versione che congela e provarla con 10^8 . Poi riscriverla con `SwingWorker`, barra di avanzamento e un pulsante "Annulla" che chiama `cancel(true)`: alla pressione l'etichetta deve dire "annullato" e il pulsante "Conta" tornare attivo.

Esercizio 4.8: Diecimila attese

Eseguire diecimila compiti che dormono un secondo con `newFixedThreadPool(200)`, con `newCachedThreadPool()` e con `newVirtualThreadPerTaskExecutor()`, misurando il tempo. Spiegare i tre risultati; poi sostituire `pausa(1000)` con un calcolo di un secondo (contare i primi fino a un milione, più volte) e ripetere: che cosa cambia, e perché?

Soluzioni

Soluzione 4.1. Quasi certamente fine, B, A: main non aspetta nessuno e stampa per primo; B dorme meno di A. Ma nessuna delle tre posizioni è garantita: fine potrebbe uscire dopo B se main venisse sospeso subito dopo i due start, e su una macchina molto carica anche A e B potrebbero invertirsi, perché sleep garantisce un'attesa minima, non un ordine. Per imporlo:

```
a.start(); a.join();
b.start(); b.join();
System.out.println("fine");
```

Così però i due thread non girano più insieme: ogni ordine imposto è parallelismo tolto. Se si vuole che A e B lavorino in parallelo e solo fine aspetti entrambi: a.start(); b.start(); a.join(); b.join(); e l'ordine fra A e B resta libero.

Soluzione 4.2.

```
class Counter1 {                                // synchronized
    private int count = 0;
    synchronized void inc() { count++; }
    synchronized int get() { return count; }
}
class Counter2 {                                // AtomicInteger
    private final AtomicInteger count = new AtomicInteger();
    void inc() { count.incrementAndGet(); }
    int get() { return count.get(); }
}
class Counter3 {                                // LongAdder
    private final LongAdder count = new LongAdder();
    void inc() { count.increment(); }
    long get() { return count.sum(); }
}
```

Nella prima anche get è synchronized: senza, una lettura potrebbe vedere un valore vecchio (visibilità). La seconda è la scelta normale: nessun lock, nessun thread bloccato, lettura esatta in ogni istante. La terza restituisce un long e la sua lettura, durante gli aggiornamenti, può non includere gli ultimi: va bene per statistiche aggiornate da moltissimi thread, come il numero di richieste servite, non per un valore che deve essere esatto adesso. La prima si sceglie quando il contatore è uno dei campi da aggiornare insieme ad altri: l'atomica protegge una variabile, il monitor protegge uno stato.

Soluzione 4.3. I numeri dipendono dalla macchina; la forma no. L'accelerazione cresce quasi linearmente fino al numero di core *fisici* veloci, rallenta sui core efficienti o sui thread logici (*hyper-threading*: due thread sullo stesso core si dividono le unità di calcolo), e oltre il numero di core resta ferma o peggiora leggermente per i context switch. La parte sequenziale (dividere, sommare, avviare i thread) è piccola ma c'è, e con la legge di Amdahl mette un tetto. Con $nTask = nThread$ i compiti non sono bilanciati: l'ultimo intervallo contiene i numeri più grandi, per cui `isPrime` scorre più divisori, e il thread che lo riceve finisce molto dopo gli altri, che intanto sono fermi. Con 64 compiti, chi finisce prende il successivo e lo sbilanciamento si riduce a quello di un singolo compito.

Soluzione 4.4.

```
class Conto {
    private final int id; private int saldo;
    Conto(int id, int saldo) { this.id = id; this.saldo = saldo; }
    int id() { return id; }
    int saldo() { return saldo; }
    void preleva(int x) { saldo -= x; }
    void versa(int x) { saldo += x; }
}
```

Con la prima trasferisci e due thread che chiamano in ciclo `trasferisci(x, y, 1)` e `trasferisci(y, x, 1)`, il blocco arriva in una frazione di secondo: `jstack` mostra i due thread in `BLOCKED`, ciascuno “waiting to lock” il monitor che l'altro “locked”. La versione con primo e secondo scelti per `id` crescente non si blocca mai: i due thread chiedono sempre i monitor nello stesso ordine, quindi il secondo aspetta che il primo abbia finito e non può esserci ciclo. La somma `x.saldo() + y.saldo()`, letta tenendo entrambi i monitor, resta uguale all'iniziale; letta senza monitor potrebbe cogliere un trasferimento a metà.

Soluzione 4.5. Con la coda bloccante:

```
BlockingQueue<Integer> coda = new ArrayBlockingQueue<>(10);
Thread produttore = new Thread(() -> {
    try {
        for (int i = 1; i <= 1000; i++) coda.put(i);
        coda.put(-1); // sentinella: non arriva altro
    } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
});
Thread consumatore = new Thread(() -> {
    try {
        long somma = 0;
        for (int v; (v = coda.take()) != -1; ) somma += v;
        System.out.println("somma: " + somma); // 500500
    } catch (InterruptedException e) { Thread.currentThread().interrupt(); }
});
produttore.start(); consumatore.start();
```

Con il Buffer a monitor il codice dei due thread è identico, con `metti` e `prendi` al posto di `put` e `take` (e un `Buffer<Integer>` generico). La fine si segnala con un valore *sentinella* che non può essere un dato (`-1` qui); con più consumatori se ne mette uno per ciascuno, oppure il consumatore che lo trova lo rimette in coda per gli altri.

Soluzione 4.6. Con `scheduleAtFixedRate` gli inizi sono a 0, 3, 6, 9 secondi: il periodo è di un secondo ma il compito ne dura tre, e con un solo thread l'esecuzione successiva parte appena quella in corso finisce, senza recuperare i turni persi. Con `scheduleWithFixedDelay` a 0, 4, 8, 12: tre di lavoro e uno di pausa. Con due thread e `scheduleAtFixedRate` non cambia niente: la libreria non sovrappone mai due esecuzioni dello stesso compito periodico, il secondo thread resta libero per altri compiti.

Soluzione 4.7. Rispetto alla versione della sezione 4.9 cambiano tre cose. Il worker va tenuto in un campo della finestra, perché il listener di “Annulla” deve raggiungerlo:

```
private SwingWorker<Long, Void> worker;
...
btnAnnulla.addActionListener(e -> {
    if (worker != null) worker.cancel(true);
});
```

In `doInBackground` il ciclo controlla già `isCancelled()` a ogni blocco; se il calcolo fosse un blocco unico, bisognerebbe controllare la bandiera di interruzione dentro il ciclo di `isPrime`, perché `cancel(true)` interrompe il thread ma è il compito che deve accorgersene. In `done()`, `get()` lancia `CancellationException`, già gestita: l'etichetta dice “annullato” e il pulsante “Conta” torna attivo nel `finally` o dopo il `try`. Da notare che `done()` viene chiamato anche dopo l'annullamento: è lì che si rimette a posto l'interfaccia, non nel listener di “Annulla”.

Soluzione 4.8. Sul portatile della sezione 4.4.4: pool da 200, 50 secondi; pool cached, 1,9 secondi; thread virtuali, 1,0 secondi. Il pool fisso serve 200 compiti alla volta, e diecimila sono cinquanta turni. Il pool cached crea diecimila thread di piattaforma, che il sistema regge (10 GB di stack *virtuale*, ma riservato, non usato) e che dormono tutti insieme: funziona, al costo di diecimila thread del kernel e del tempo per crearli. I thread virtuali dormono senza occupare nulla. Con un compito di calcolo le tre versioni si avvicinano: diecimila secondi di lavoro su n core durano $10000/n$ secondi comunque, e il pool fisso con tanti thread quanti i core è la versione più efficiente delle tre, perché non paga la creazione di thread né i context switch fra migliaia di thread che vorrebbero tutti calcolare. I thread virtuali risolvono l'attesa, non il calcolo.